

THE GRID CATALOGUE

The lecture is a story. This is the lookup table. Keep it open while you build.

WHAT IS IN HERE

Every track keyword, every placement form, and the alignment matrix in full.

HOW TO READ IT

Left column is what you write. Right column is what the browser does with it.

WHEN TO OPEN IT

Mid-build, when a track list does what you did not ask for.

FLEXBOX REASONS ABOUT ONE AXIS AT A TIME

WHAT YOU ASKED FOR

A sidebar, a main column and an aside that line up as a **row** — and a masthead and footer that line up with them as a **column**.

WHAT FLEXBOX HEARD

A row. Then, separately, another row. Nothing connects the two, because a flex container has no concept of the column its items share with anything outside it.

WHAT GRID IS

A layout system that holds **both** directions at once, in the container, before any item has an opinion.

TRACKS AND LINES

EIGHT LINES OF CSS, AND NO WRAPPER ELEMENTS

the same five regions, in Grid

↺ reload

open ↗

Show me the CSS that does this

THE SAME FIVE REGIONS, AND NO WRAPPER ELEMENTS

masthead

sidebar

main

The main region holds more than the others, exactly as before. Nothing about that is a problem any more.

aside

footer

Both gutters are 12px. The sidebar meets the footer exactly.

```
container inner width  1498px
the grid wants        1498px
gutter, sidebar to main 12px
sidebar bottom to footer 12px
wrapper elements needed 0
the browser resolved 1fr 2fr 1fr to
                        368.5px 737px 368.5px
```

► **HANDS-ON** [demos/01-why-grid/02-grid/](#) — ask them to guess the line count **BEFORE** you press the button.

YOU DEFINE THE CONTAINER FIRST

what it is A two-dimensional layout system, declared on the **container**.

key trait The tracks exist before any item does. Items then land in them, or are told where to go.

use cases Page shells. Card rows. Anything with rows *and* columns.

tip `display: grid` alone gives you **one column**, which looks exactly like nothing happened.

```
1 .page {  
2   display: grid;  
3   grid-template-columns: 1fr 2fr 1fr;  
4   grid-template-rows: auto 1fr auto;  
5   gap: 16px;  
6 }  
7  
8 .masthead {  
9   grid-column: 1 / -1;  
10 }
```

THE LAYOUT PANE

WHERE

Elements · the `grid` badge

Layout · *Show line numbers*

Layout · *Show line names* · *Hide line labels*

Layout · *Show track sizes*

Layout · *Show area names*

Layout · *Extend grid lines*

WHAT IT SHOWS

Next to any element with `display: grid` or `inline-grid`. Click it to toggle the overlay on the page

The default: every line numbered from the start and, negatively, from the end — where `-1` comes from

Named lines instead of numbers, or no labels at all

What you wrote and what the browser computed, side by side — `1fr` next to `560px`

The picture you drew with `grid-template-areas`, written on the page

The lines run to the edge of the viewport, for checking alignment across regions

► **ALSO** — every grid on the page is listed in the Layout pane with its own overlay colour. Flex containers get a `flex` badge the same way.

THE TOOL YOU WILL USE ALL EVENING

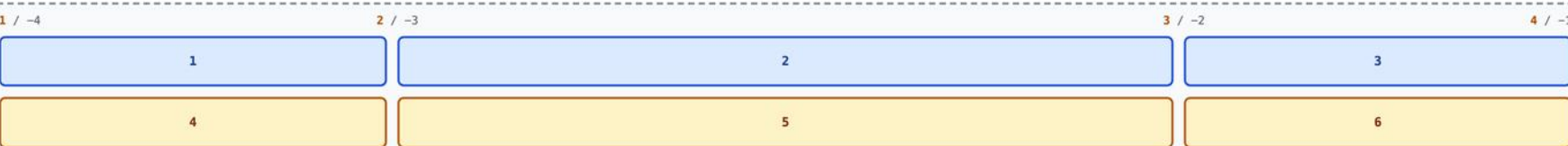
demos/02-tracks — type a track list, watch the browser answer reload open

200px 200px 200px 33% 33% 33% 1fr 1fr 1fr **1fr 2fr 1fr** auto 400px 1fr repeat(3, 1fr) repeat(4, minmax(120px, 1fr)) min-content max-content 1fr

grid-template-columns 1fr 2fr 1fr

gap 10px items 6

THE TRACKS, AND THE LINES THAT COME WITH THEM



3 tracks, so 4 lines, numbered 1 to 4.

you wrote 1fr 2fr 1fr
browser used 370px 740px 370px
gap 10px
items 6 in 2 rows

Read the second line, not the first. That is what the layout actually resolved to, in pixels — the USED value. Every track you wrote as a

► **HANDS-ON** demos/02-tracks/ — start on 200px 200px 200px and resize the window.

TRACK SIZING

VALUE	MEANS	REACH FOR IT WHEN
<code>200px</code>	exactly that	the thing has a real size — an icon, a control
<code>25%</code>	a share of the container, ignoring <code>gap</code>	almost never in a grid. Use <code>fr</code>
<code>1fr</code>	a share of what is <i>left over</i>	the default answer for a flexible column
<code>auto</code>	content size, then stretches into spare room	a label column beside a field
<code>min-content</code>	as narrow as possible without overflowing	a column that should hug its longest word
<code>max-content</code>	as wide as it wants, wrapping nothing	rarely, and never for prose
<code>minmax(a, b)</code>	a floor and a ceiling	whenever a track has both a limit and freedom
<code>fit-content(a)</code>	content size, capped at <code>a</code>	a caption column that must not run away

THE SIZING VOCABULARY, AS ARITHMETIC

Fixed tracks and gaps are paid first. `fr` shares what is left — and at a narrow width there may be nothing left.

TEMPLATE, GAP: 16PX	AT 1000PX	AT 400PX
<code>200px 1fr 200px</code>	$200 \cdot 568 \cdot 200$	$200 \cdot 0 \cdot 200$ — 432 wide, overflows
<code>repeat(3, 1fr 140px)</code>	six tracks: 166.67 and 140, alternating	the three <code>fr</code> tracks are 0 — 500 wide
<code>repeat(3, minmax(10rem, 1fr))</code>	322.67×3	160×3 — the floor holds, 512 wide

`repeat()` takes a **pattern**, not one track. A floor that does not fit is an overflow, not a squeeze — which is the reason `auto-fit` exists.



SIZING

fr IS NOT PERCENT WITH A NICER NAME

same intent, two units, one gap slider

↺ reload

open ↗

gap  0px ☐ trim the percentages to 32%

grid-template-columns: 33.333% 33.333% 33.333%



grid-template-columns: 1fr 1fr 1fr



container inner width 1498px

gap x 2 0px

percent resolved to 499.328px 499.328px 499.328px
and the row then wants 1498px (0)

fr resolved to 499.328px 499.328px 499.344px

With no gap these are the same layout, and you should say so plainly.
Nothing so far distinguishes the two units.

Now drag the gap. Every real design has one.

► **HANDS-ON** [demos/03-fr-vs-percent/](#) — gap at 0, then drag it in one motion and stop talking.

YOU HAVE MET THIS BUG. IT IS THE SAME BUG.

why 1fr overflows, and what minmax(0, 1fr) fixes

[↶ reload](#)[open ↷](#)

☐ paste a long URL into the content ☐ try overflow: hidden ☐ try a narrower sidebar

grid-template-columns: 200px 1fr

sidebar

Ordinary text wraps, so nothing is wrong until something cannot wrap.

grid-template-columns: 200px minmax(0, 1fr)

sidebar

Ordinary text wraps, so nothing is wrong until something cannot wrap.

```
container inner width    1498px
200px 1fr               wants 1498px (0)
200px minmax(0, 1fr)    wants 1498px
resolved, top grid       200px 1288px
resolved, bottom grid    200px 1288px
```

Nothing is wrong yet, and nothing will be while the content can wrap.
That is why this bug reaches production: it needs a URL, a long token,
a wide `<pre>` or a table, and none of those are in anybody's test data.

Tick the first box.

► **HANDS-ON** [demos/07-minmax-0/](#) — take the two wrong fixes seriously before you give the right one.

WHY `1fr` HAS A FLOOR, AND THREE WAYS TO LOWER IT

`1fr` is `minmax(auto, 1fr)`. For a track, an `auto` minimum asks each item for its **automatic minimum size** — its content — unless the item is a scroll container. One unbreakable string, and the column refuses to shrink.

`overflow-wrap: break-word` does not change that: it breaks the word only after layout. `anywhere` does, because its break points count toward min-content.

AT 768, CYCLE 1 WITH THE URL

TRACKS

`1fr 2fr 1fr`

108 · 561 · 112, page scrolls 69px

`min-width: 0` on the item

172 · 344 · 172

`overflow: hidden` on the item

172 · 344 · 172, and it clips

`minmax(0, 2fr)` on the track

172 · 344 · 172, for every item

REPEAT, AUTO-FIT AND AUTO-FILL

WRITTEN

```
repeat(3, 1fr)
```

```
repeat(2, 1fr 140px)
```

```
repeat(auto-fit, minmax(15rem, 1fr))
```

```
repeat(auto-fill, minmax(15rem, 1fr))
```

PRODUCES

three equal columns, always three

four tracks — `repeat()` takes a **pattern**

as many as fit; **empty tracks collapse** and the items grow

as many as fit; **empty tracks stay** at their size

THEY ARE IDENTICAL...

...whenever there are enough items to fill the row. That is why so many people believe they are the same keyword.

CHOOSING

`auto-fit` for a card row. `auto-fill` when an empty slot is information — a calendar, a seating chart.

THE DIFFERENCE, AS A NUMBER

First the browser counts tracks: `floor((W + gap) / (min + gap))`.

Only then does the keyword decide what happens to the empty ones.

3 ITEMS, <code>MINMAX(11REM, 1FR)</code> , GAP 16	TRACKS	AUTO-FILL ITEM	AUTO-FIT ITEM
400px	2	192	192 — identical
800px	4	188, one empty slot	256 , the empty track is 0
1152px	6	178.66, three empty slots	373.33 , three tracks at 0

► **WHAT DEVTOOLS SHOWS** — under `auto-fit` the computed `grid-template-columns` still lists the collapsed tracks, as `0px`. Six tracks, three of them zero.

LIVE CODING — MAKE THE CARD ROW BREATHE

- 1 Start from `repeat(3, 1fr)`. Narrow the window. It is awful.
- 2 Swap to `repeat(auto-fit, minmax(15rem, 1fr))`.
- 3 Add a long unbroken word to one card. Watch it burst.
- 4 Fix it with `minmax(0, 1fr)` on the parent track.
- 5 Add a hover with `transform`, and a reduced-motion query.

► **NOW** — this became cycle 3, `live-code/03-card-row-no-query`.

PLACEMENT

PUTTING AN ITEM SOMEWHERE ON PURPOSE

demos/05-placement/01-lines

↻ reload

open ↗

column-start 1 column-end 3 row-start 1 row-end 2 column-end = -1 toggle 4 / 6 columns

A 4 X 3 GRID. ONE ITEM IS PLACED; THE REST FLOW AROUND IT.



```
.placed {  
  grid-column: 1 / 3;  
  grid-row: 1 / 2;  
}
```

```
/* the same placement, written with span */  
.placed {  
  grid-column: 1 / span 2;  
}
```

► **HANDS-ON** [demos/05-placement/01-lines/](#) — set 1 / 2 and ask why it covers one column.

FOUR WAYS TO SAY THE SAME PLACEMENT

```
1  .a {  
2    grid-column: 1 / 3;  
3    grid-row: 2 / 4;  
4  }  
5  
6  .b {  
7    grid-column: 1 / span 2;  
8    grid-row: 2 / span 2;  
9  }  
10  
11 .c {  
12   /* row-start / column-start / row-end / column-end */  
13   grid-area: 2 / 1 / 4 / 3;  
14 }  
15  
16 .d {  
17   grid-column: 1 / -1;  
18   /* all the way across, whatever the track count */  
19 }
```

PLACEMENT

WRITTEN	MEANS
<code>grid-column: 1 / 3</code>	from line 1 to line 3 — that is two columns
<code>grid-column: 1 / span 2</code>	the same thing, counted as a width
<code>grid-column: 1 / -1</code>	all the way across, whatever the track count
<code>grid-row: 2 / 4</code>	from row line 2 to row line 4
<code>grid-area: 2 / 1 / 4 / 3</code>	row-start / column-start / row-end / column-end
<code>grid-area: main</code>	the area called main in the container's picture

► **THREE TRACKS MEANS FOUR LINES.** You count the edges, not the things.

A CHEQUER'S PERIOD, WORKED

`:nth-child(an + b)` matches positions `b`, `a + b`, `2a + b` ... with `n` counting from 0. `odd` is `2n + 1`.

On a board `w` tiles wide, the dark tiles of row one are 1, 3, 5... and the dark tiles of row two are shifted by one. The pattern comes back after **two rows**: the period is `2w`.

If `w` is **odd**, every row already starts on the other parity, and `odd` is a chequer. If `w` is **even**, `odd` is stripes.

```
/* w = 4, so the period is 8 */
.tiles > :nth-child(8n + 1), /* row 1 */
.tiles > :nth-child(8n + 3),
.tiles > :nth-child(8n + 6), /* row 2 */
.tiles > :nth-child(8n + 8) {
  background: var(--dark);
}

/* w = 5: odd is already a chequer */
.five > :nth-child(odd) { ... }
```

OR DRAW THE LAYOUT AND LET THE ITEMS CLAIM A NAME

demos/05-placement/02-areas – the picture is the layout

reloadopen

desktoptabletphonea gap in the gridnot a rectangleagged rows

grid-template-areas

"masthead masthead masthead masthead"
"sidebar main main aside"
"sidebar main main aside"
"footer footer footer footer"

THE LAYOUT THAT PICTURE PRODUCES

masthead

sidebarmainaside

footer

Valid. Every row has the same number of cells and every named area is a rectangle.

A dot is a deliberately empty cell – useful, and the reason "a gap in the grid" is a preset here rather than a mistake.

Note what the five regions themselves say: `grid-area: main` and nothing else. The picture holds the layout; the items only claim a name. That is what makes a responsive page shell three pictures in three media queries

► **HANDS-ON** [demos/05-placement/02-areas/](#) — edit a row, then press **PHONE**, then press **NOT A RECTANGLE**.

NAMED AREAS

on the container

`grid-template-areas`, one quoted string per row.

on each item

`grid-area: <name>` and nothing else.

a dot

`.` is a deliberately empty cell.

rule one

Every row needs the **same number of cells**.

rule two

Every named area must be a **rectangle**. There is no L shape.

when you break either

The declaration is dropped in silence and you get auto-placement.

```
.page {  
  grid-template-areas:  
    "head head head"  
    "side main aside"  
    "foot foot foot";  
}  
  
.masthead { grid-area: head; }  
.sidebar { grid-area: side; }
```

LIVE CODING — THE PAGE SHELL, FROM NOTHING

- 1 Five regions, stacked. One column, five names, no media query.
- 2 Give each region its `grid-area`. Names only, no numbers.
- 3 One breakpoint: two columns. Rewrite *only* the picture.
- 4 Second breakpoint: three columns. Same again.
- 5 Break it on purpose — make one row short a cell.

► **NOW** — this became cycle 5, `live-code/05-draw-then-mobile-first`.

THE IMPLICIT GRID

PROPERTY	DOES
<code>grid-auto-rows</code>	sizes rows nobody declared
<code>grid-auto-columns</code>	the same for columns
<code>grid-auto-flow: row</code>	the default — fill along rows
<code>grid-auto-flow: column</code>	fill along columns instead
<code>grid-auto-flow: dense</code>	let a later, smaller item back-fill a hole

THE COST OF `dense`

It changes the **visual** order without touching the DOM, so what people see and what they tab through stop matching. Fine for independent cards. A bug in a form.

WHY IMPLICIT ROWS MATTER

A list from data has an unknown length. The implicit grid is what makes that work without counting anything.

ALIGNMENT, IN BOTH DIRECTIONS AT ONCE

THE TRACKS INSIDE THE CONTAINER

`justify-content` ·
`align-content`

Used when the tracks are narrower than the container.

THE ITEMS INSIDE THEIR CELLS

`justify-items` · `align-items`

And `justify-self` /
`align-self` for one item.

THE GOTCHA FROM WEEK 4

In Flexbox, `justify-` follows `flex-direction`. In Grid it is **always** the inline axis. It does not turn.

► **SHORTHAND** — `place-items: center` is both.

ALIGNMENT — THE WHOLE MATRIX

PROPERTY	ALIGNS	ALONG
<code>justify-content</code>	the tracks inside the container	inline (horizontal)
<code>align-content</code>	the tracks inside the container	block (vertical)
<code>justify-items</code>	every item inside its cell	inline
<code>align-items</code>	every item inside its cell	block
<code>justify-self</code>	one item inside its cell	inline
<code>align-self</code>	one item inside its cell	block
<code>place-items</code>	<code>align-items</code> then <code>justify-items</code>	
<code>place-content</code>	<code>align-content</code> then <code>justify-content</code>	

► **UNLIKE FLEXBOX** — these axes never swap. `justify-` is always inline, whatever else you change.

GRID OR FLEX

THE SAME CARD ROW, THREE WAYS

demos/06-three-ways — one width slider, three containers

↻ reload

open ↗

container width  760px cards  6

FLEXBOX — flex: 1 1 200px; flex-wrap: wrap

Latency

A slightly longer line of body copy.

[view](#)**Errors**

A shorter note.

[view](#)**Traffic**

A slightly longer line of body copy.

[view](#)**Saturation**

A slightly longer line of body copy.

[view](#)**Uptime**

A shorter note.

[view](#)**Cost**

A slightly longer line of body copy.

[view](#)

GRID — repeat(3, 1fr)

Latency

A slightly longer line of body copy.

[view](#)**Errors**

A shorter note.

[view](#)**Traffic**

A slightly longer line of body copy.

[view](#)**Saturation**

A slightly longer line of body copy.

[view](#)**Uptime**

A shorter note.

[view](#)**Cost**

A slightly longer line of body copy.

[view](#)

► **HANDS-ON** demos/06-three-ways/ — find a width where all three agree, then break it.

GRID OR FLEX

YOU ARE BUILDING	USE	BECAUSE
a page shell	Grid	rows and columns must line up together
a card row of unknown length	Grid	<code>auto-fit</code> removes the media query
a dashboard, a gallery	Grid	two dimensions, and items that span
a navbar	Flex	one axis, sizes from content
a row of buttons	Flex	one axis. Grid here is the classic overreach
the inside of a card	Flex	a column, with one child taking the slack
a chip with an icon	Flex	two things, centred, on one axis
a form with a label column	Grid	the labels must align across rows

You don't have to choose one over the other — Flexbox can be used inside a grid cell.

RESPONSIVE

NOW, THE WIDTH YOU DID NOT DESIGN FOR

Everything so far adapts **by itself**.

`fr`, `minmax` and `auto-fit` got
you a long way with no query at all.

A media query is for the decisions the content cannot make: **this is a phone, so put the nav away.**

RESPONSIVE — THE UNITS

UNIT	RELATIVE TO	USE FOR
<code>rem</code>	the root font size — the reader's own setting	type, spacing, breakpoints
<code>em</code>	the element's own font size	a media query, so it moves with the reader
<code>ch</code>	the width of a "0"	<code>max-width</code> on prose
<code>vw</code>	1% of the viewport width	inside <code>clamp()</code> , never alone
<code>dvh</code>	the viewport height <i>as it is now</i>	full-height layouts on a phone
<code>vh</code>	the largest viewport height	rarely — the address bar will cover the bottom

`clamp()`, AND THE ACCESSIBILITY TRAP

RIGHT

```
h1 {  
  font-size: clamp(1.6rem, 1rem + 2.2vw, 2.6rem);  
}
```

The `rem` term keeps the reader's font setting in the answer.

WRONG

```
h1 {  
  font-size: clamp(1.6rem, 4.2vw, 2.6rem);  
}
```

Pure `vw`. Somebody who set a larger default font is ignored entirely.

► **CLAMP(MIN, PREFERRED, MAX)** returns the middle one unless it falls outside the other two.

IMAGES

TOOL	JOB
<code>aspect-ratio</code>	decides how big the box is, and reserves it before the file arrives
<code>width / height</code> attributes	the same reservation, for free, before the CSS loads
<code>object-fit: cover</code>	fills the box and crops. For photographs
<code>object-fit: contain</code>	fits inside and leaves space. For logos and diagrams
<code>srcset="... 800w"</code>	the candidate files, each labelled with its real width
<code>sizes="(min-width: 60em) 33vw, 100vw"</code>	how wide the slot will be, so the browser can choose before layout

► **TWO PROPERTIES, TWO JOBS.** `aspect-ratio` sizes the box; `object-fit` handles the picture inside it.

aspect-ratio AND THE WIDTH AND HEIGHT ATTRIBUTES

`<img width="600" height="400">` is not a size any more. The browser maps the two attributes to `aspect-ratio: auto 600 / 400`, so with `width: 100%` and `height: auto` the box is reserved before the file arrives, and nothing jumps.

`auto` means "the picture's own ratio once it loads". A bare `aspect-ratio: 3 / 2` overrides it: every box is 3 : 2, whatever the file.

At least one of `width` and `height` must be automatic, or `aspect-ratio` does nothing. Baseline since September 2021; widely available since March 2024.

OBJECT-FIT

THE PICTURE INSIDE THE BOX

<code>fill</code> (default)	stretched to the box — distorted
<code>cover</code>	fills the box, cropped
<code>contain</code>	all of it, with empty bands
<code>none</code> · <code>scale-down</code>	natural size · the smaller of none and contain

`object-position` chooses which part survives the crop.

srcset AND sizes, BRIEFLY

```
1 
```

`sizes` is a media condition and not a selector, because the browser has to choose a file **before** layout has happened.

MOTION

CHEAP

`transform` · `opacity` — and only these two

The compositor moves an existing layer. No layout, no paint.

EXPENSIVE

`width` · `height` · `top` · `left` · `margin` · `padding`: layout and paint, every frame.

`color` · `background` · `box-shadow`: no layout, but a repaint every frame — a shadow's blur is the slow kind.

TRANSITION: ALL

Covers whatever happens to change — including the expensive ones. Nobody who writes it knows what it covered.

```
@media (prefers-reduced-motion: reduce) {  
  :root { --duration: 0ms; }  
}
```

THE QUERY A COMPONENT ASKS ABOUT ITSELF

A media query asks about the **window**. But a card does not care how wide the window is — it cares how wide *it* is, and the same card might sit in a sidebar and in a full-width row on the same page.

Container queries ask the second question. **Not examined, not needed for the assignment** — but it is where this is going, and it is the reason "breakpoints come from content" is more than a slogan.

`container-type: inline-size` makes an element a query container; `@container` asks about the nearest one; `cqi` is 1% of its inline size. In all three browser engines since February 2023.

```
1 .panel {  
2   container-type: inline-size;  
3 }  
4  
5 @container (min-width: 30em) {  
6   .card {  
7     grid-template-columns: 8rem 1fr;  
8   }  
9 }
```

THE FIVE MISTAKES

THE MISTAKE

Grid inside a button row

`auto-fit` and `auto-fill` are the same

Desktop-first breakpoints

Fixed heights on layout regions

Animating `width` or `left`

WHAT IS TRUE

One axis is Flexbox. Grid is for the page

Identical only while there are enough items. With fewer, fit collapses and fill does not

`min-width` accumulates; `max-width` has to undo itself

A region is as tall as its content. Fixed heights break in another language

Layout and paint every frame. Use `transform`

► **AND THE SIXTH** — `1fr` is `minmax(auto, 1fr)`, so it will not shrink below its content. `minmax(0, 1fr)` on the track.

WHERE TO LOOK IT UP

IN THIS REPO

`demos/02-tracks/` — type a track list, see what the browser did.

`demos/04-auto-fit-fill/` — the one everybody gets wrong.

IN THE BROWSER

DevTools, select a container, click the `grid` badge. It draws the lines, the numbers and the gaps. The Layout pane has the rest.

ELSEWHERE

MDN's CSS grid layout guide. And CSS Grid Garden — twenty minutes, and it sticks.

TONIGHT, AND THIS WEEK

THE CLASS ASSIGNMENT

A painting and a chessboard as warm-ups, then a dashboard that lays itself out three ways.

One rule: not a single `max-width` query.

HOME ASSIGNMENT 2

Rebuild a real page from a screenshot. Grid and Flex, mobile-first.

Due before lesson 7.

BEFORE NEXT WEEK

Nothing to install. Next week is Tailwind, and it is all of this with new names.

Optional: CSS Grid Garden, twenty minutes.